

ExportDB#

<https://pytorch.org/docs/stable/generated/exportdb/index.html>

Description:

ExportDB is a centralized dataset of supported and unsupported export cases. It is targeted towards users who want to understand specifically what types of code are supported, the subtleties of export, and how to modify their existing code to be compatible with export. Note that this is not an exhaustive set of everything that is supported by exportdb, but it covers the most common and confusing use cases that users will run into. If you have a feature that you think needs a stronger guarantee from us to support in export please create an issue in the [pytorch/pytorch](#) repo with a `module:export` tag. Tags: `torch.escape-hatch`

Code Examples

```
# mypy: allow-untyped-defs
import torch
import torch._dynamo as torchdynamo

class AssumeConstantResult(torch.nn.Module):
    """
    Applying `assume_constant_result` decorator to burn make
    non-tracable code as constant.
    """

    @torchdynamo.assume_constant_result
    def get_item(self, y):
        return y.int().item()

    def forward(self, x, y):
        return x[: self.get_item(y)]

example_args = (torch.randn(3, 2), torch.tensor(4))
tags = {"torch.escape-hatch"}
model = AssumeConstantResult()

torch.export.export(model, example_args)
```

ExportedProgram:

```
class GraphModule(torch.nn.Module):  
    def forward(self, x: "f32[3, 2]", y: "i64[]"):  
        slice_1: "f32[3, 2]" =  
torch.ops.aten.slice.Tensor(x, 0, 0, 4);  x = None  
        return (slice_1,)
```

Graph signature:

```
# inputs  
x: USER_INPUT  
y: USER_INPUT  
  
# outputs  
slice_1: USER_OUTPUT
```

Range constraints: {}

```

# mypy: allow-untyped-defs
import torch

class MyAutogradFunction(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x):
        return x.clone()

    @staticmethod
    def backward(ctx, grad_output):
        return grad_output + 1

class AutogradFunction(torch.nn.Module):
    """
    TorchDynamo does not keep track of backward() on autograd
    functions. We recommend to
    use `allow_in_graph` to mitigate this problem.
    """

    def forward(self, x):
        return MyAutogradFunction.apply(x)

example_args = (torch.randn(3, 2),)
model = AutogradFunction()

torch.export.export(model, example_args)

```

ExportedProgram:

```
class GraphModule(torch.nn.Module):  
    def forward(self, x: "f32[3, 2]"):   
        clone: "f32[3, 2]" =   
torch.ops.aten.clone.default(x);  x = None  
        return (clone,)
```

Graph signature:

```
# inputs  
x: USER_INPUT  
  
# outputs  
clone: USER_OUTPUT
```

Range constraints: {}

```

# mypy: allow-untyped-defs
import torch

class ClassMethod(torch.nn.Module):
    """
    Class methods are inlined during tracing.
    """

    @classmethod
    def method(cls, x):
        return x + 1

    def __init__(self) -> None:
        super().__init__()
        self.linear = torch.nn.Linear(4, 2)

    def forward(self, x):
        x = self.linear(x)
        return self.method(x) * self.__class__.method(x) *
type(self).method(x)

example_args = (torch.randn(3, 4),)
model = ClassMethod()

torch.export.export(model, example_args)

```

ExportedProgram:

```
class GraphModule(torch.nn.Module):
    def forward(self, p_linear_weight: "f32[2, 4]",
p_linear_bias: "f32[2]", x: "f32[3, 4]"):
        linear: "f32[3, 2]" =
torch.ops.aten.linear.default(x, p_linear_weight,
p_linear_bias); x = p_linear_weight = p_linear_bias = None

        add: "f32[3, 2]" =
torch.ops.aten.add.Tensor(linear, 1)
        add_1: "f32[3, 2]" =
torch.ops.aten.add.Tensor(linear, 1)

        mul: "f32[3, 2]" =
torch.ops.aten.mul.Tensor(add, add_1); add = add_1 = None

        add_2: "f32[3, 2]" =
torch.ops.aten.add.Tensor(linear, 1); linear = None

        mul_1: "f32[3, 2]" =
torch.ops.aten.mul.Tensor(mul, add_2); mul = add_2 = None
        return (mul_1,)
```

Graph signature:

```
# inputs
p_linear_weight: PARAMETER target='linear.weight'
p_linear_bias: PARAMETER target='linear.bias'
x: USER_INPUT

# outputs
mul_1: USER_OUTPUT
```

Range constraints: {}

```

# mypy: allow-untyped-defs
import torch

from functorch.experimental.control_flow import cond

class MySubModule(torch.nn.Module):
    def foo(self, x):
        return x.cos()

    def forward(self, x):
        return self.foo(x)

class CondBranchClassMethod(torch.nn.Module):
    """
    The branch functions (`true_fn` and `false_fn`) passed to
    cond() must follow these rules:
        - both branches must take the same args, which must also
        match the branch args passed to cond.
        - both branches must return a single tensor
        - returned tensor must have the same tensor metadata, e.g.
        shape and dtype
        - branch function can be free function, nested function,
        lambda, class methods
        - branch function can not have closure variables
        - no inplace mutations on inputs or global variables

    This example demonstrates using class method in cond().

    NOTE: If the `pred` is test on a dim with batch size < 2, it
    will be specialized.
    """

    def __init__(self) -> None:
        super().__init__()
        self.subm = MySubModule()

```



```

def bar(self, x):
    return x.sin()

def forward(self, x):
    return cond(x.shape[0] <= 2, self.subm.forward,
self.bar, [x])

example_args = (torch.randn(3),)
tags = {
    "torch.cond",
    "torch.dynamic-shape",
}
model = CondBranchClassMethod()

torch.export.export(model, example_args)

```

```

ExportedProgram:
class GraphModule(torch.nn.Module):
    def forward(self, x: "f32[3]"):
        sin: "f32[3]" = torch.ops.aten.sin.default(x);
x = None
        return (sin,)

Graph signature:
# inputs
x: USER_INPUT

# outputs
sin: USER_OUTPUT

Range constraints: {}

```

```

# mypy: allow-untyped-defs
import torch

from functorch.experimental.control_flow import cond

class CondBranchNestedFunction(torch.nn.Module):
    """
    The branch functions (`true_fn` and `false_fn`) passed to
    cond() must follow these rules:
        - both branches must take the same args, which must also
        match the branch args passed to cond.
        - both branches must return a single tensor
        - returned tensor must have the same tensor metadata, e.g.
        shape and dtype
        - branch function can be free function, nested function,
        lambda, class methods
        - branch function can not have closure variables
        - no inplace mutations on inputs or global variables

    This example demonstrates using nested function in cond().

    NOTE: If the `pred` is test on a dim with batch size < 2, it
    will be specialized.
    """

    def forward(self, x):
        def true_fn(x):
            def inner_true_fn(y):
                return x + y

            return inner_true_fn(x)

        def false_fn(x):
            def inner_false_fn(y):
                return x - y

            return inner_false_fn(x)

```

```

        return cond(x.shape[0] < 10, true_fn, false_fn, [x])

example_args = (torch.randn(3),)
tags = {
    "torch.cond",
    "torch.dynamic-shape",
}
model = CondBranchNestedFunction()

torch.export.export(model, example_args)

```

```

ExportedProgram:
    class GraphModule(torch.nn.Module):
        def forward(self, x: "f32[3]"):
            add: "f32[3]" = torch.ops.aten.add.Tensor(x,
x);  x = None
            return (add,)

Graph signature:
    # inputs
    x: USER_INPUT

    # outputs
    add: USER_OUTPUT

Range constraints: {}

```

```

# mypy: allow-untyped-defs
import torch

from functorch.experimental.control_flow import cond

```

```
class CondBranchNonlocalVariables(torch.nn.Module):
    """
    The branch functions (`true_fn` and `false_fn`) passed to
    cond() must follow these rules:
    - both branches must take the same args, which must also
    match the branch args passed to cond.
    - both branches must return a single tensor
    - returned tensor must have the same tensor metadata, e.g.
    shape and dtype
    - branch function can be free function, nested function,
    lambda, class methods
    - branch function can not have closure variables
    - no inplace mutations on inputs or global variables
    """
```

This example demonstrates how to rewrite code to avoid capturing closure variables in branch functions.

The code below will not work because capturing closure variables is not supported.

```
```
my_tensor_var = x + 100
my_primitive_var = 3.14

def true_fn(y):
 nonlocal my_tensor_var, my_primitive_var
 return y + my_tensor_var + my_primitive_var

def false_fn(y):
 nonlocal my_tensor_var, my_primitive_var
 return y - my_tensor_var - my_primitive_var

return cond(x.shape[0] > 5, true_fn, false_fn, [x])
```
```

NOTE: If the `pred` is test on a dim with batch size < 2, it will be specialized.

```
"""
```

```

def forward(self, x):
    my_tensor_var = x + 100
    my_primitive_var = 3.14

    def true_fn(x, y, z):
        return x + y + z

    def false_fn(x, y, z):
        return x - y - z

    return cond(
        x.shape[0] > 5,
        true_fn,
        false_fn,
        [x, my_tensor_var, torch.tensor(my_primitive_var)],
    )

example_args = (torch.randn(6),)
tags = {
    "torch.cond",
    "torch.dynamic-shape",
}
model = CondBranchNonlocalVariables()

torch.export.export(model, example_args)

```

ExportedProgram:

```
class GraphModule(torch.nn.Module):
    def forward(self, c_lifted_tensor_0: "f32[]", x:
"f32[6]"):
        add: "f32[6]" = torch.ops.aten.add.Tensor(x,
100)

        lift_fresh_copy: "f32[]" =
torch.ops.aten.lift_fresh_copy.default(c_lifted_tensor_0);
c_lifted_tensor_0 = None
        detach_: "f32[]" =
torch.ops.aten.detach_.default(lift_fresh_copy);
lift_fresh_copy = None

        add_1: "f32[6]" = torch.ops.aten.add.Tensor(x,
add); x = add = None
        add_2: "f32[6]" = torch.ops.aten.add.Tensor(add_1,
detach_); add_1 = detach_ = None
        return (add_2,)
```

Graph signature:

```
# inputs
c_lifted_tensor_0: CONSTANT_TENSOR target='lifted_tensor_0'
x: USER_INPUT

# outputs
add_2: USER_OUTPUT
```

Range constraints: {}

Notes & Warnings

NOTE Note Tags: torch.escape-hatch Support Level: SUPPORTED

NOTE Note Tags: Support Level: SUPPORTED

NOTE Note Tags: Support Level: SUPPORTED

NOTE Note Tags: torch.cond, torch.dynamic-shape Support Level: SUPPORTED

NOTE Note Tags: torch.cond, torch.dynamic-shape Support Level: SUPPORTED

NOTE Note Tags: torch.cond, torch.dynamic-shape Support Level: SUPPORTED

NOTE Note Tags: torch.cond, python.closure Support Level: SUPPORTED

NOTE Note Tags: torch.cond, torch.dynamic-shape Support Level: SUPPORTED

NOTE Note Tags: torch.cond, torch.dynamic-shape Support Level: SUPPORTED

NOTE Note Tags: torch.dynamic-value, torch.escape-hatch Support Level: SUPPORTED

NOTE Note Tags: torch.dynamic-value, torch.escape-hatch Support Level: SUPPORTED

NOTE Note Tags: Support Level: SUPPORTED

NOTE Note Tags: python.data-structure Support Level: SUPPORTED

NOTE Note Tags: python.assert Support Level: SUPPORTED

NOTE Note Tags: torch.dynamic-shape Support Level: SUPPORTED

NOTE Note Tags: torch.dynamic-shape, python.control-flow Support Level: SUPPORTED

NOTE Note Tags: torch.dynamic-shape, torch.map Support Level: SUPPORTED

NOTE Note Tags: torch.dynamic-shape Support Level: SUPPORTED

NOTE Note Tags: torch.dynamic-shape Support Level: SUPPORTED

NOTE Note Tags: python.data-structure Support Level: SUPPORTED

NOTE Note Tags: torch.dynamic-shape, python.data-structure, python.assert
Support Level: SUPPORTED

NOTE Note Tags: python.data-structure, python.control-flow Support Level:
SUPPORTED

NOTE Note Tags: python.closure Support Level: SUPPORTED

NOTE Note Tags: python.context-manager Support Level: SUPPORTED

NOTE Note Tags: Support Level: SUPPORTED

NOTE Note Tags: torch.dynamic-shape Support Level: SUPPORTED

NOTE Note Tags: Support Level: SUPPORTED

NOTE Note Tags: python.control-flow Support Level: SUPPORTED

NOTE Note Tags: python.control-flow Support Level: SUPPORTED

NOTE Note Tags: python.builtin Support Level: SUPPORTED

NOTE Note Tags: python.builtin Support Level: SUPPORTED

NOTE Note Tags: torch.mutation Support Level: SUPPORTED

NOTE Note Tags: torch.dynamic-shape, python.builtin Support Level:
NOT_SUPPORTED_YET

NOTE

Note Tags: python.object-model Support Level: NOT_SUPPORTED_YET

NOTE

Note Tags: python.object-model Support Level: NOT_SUPPORTED_YET

NOTE

Note Tags: torch.operator Support Level: NOT_SUPPORTED_YET

Detailed Documentation

Supported#

ExportDB is a centralized dataset of supported and unsupported export cases. It is targeted towards users who want to understand specifically what types of code are supported, the subtleties of export, and how to modify their existing code to be compatible with export. Note that this is not an exhaustive set of everything that is supported by exportdb, but it covers the most common and confusing use cases that users will run into.

If you have a feature that you think needs a stronger guarantee from us to support in export please create an issue in the pytorch/pytorch repo with a module:export tag.

assume_constant_result#

Original source code:

autograd_function#

Original source code:

class_method#

Original source code:

cond_branch_class_method#

Original source code:

cond_branch_nested_function#

Original source code:

cond_branch_nonlocal_variables#

Original source code:

cond_closed_over_variable#

Original source code:

cond_operands#

Original source code:

cond_predicate#

Original source code:

constrain_as_size_example#

Original source code:

constrain_as_value_example#

Original source code:

decorator#

Original source code:

dictionary#

Original source code:

dynamic_shape_assert#

Original source code:

dynamic_shape_constructor#

Original source code:

dynamic_shape_if_guard#

Original source code:

dynamic_shape_map#

Original source code:

dynamic_shape_slicing#

Original source code:

dynamic_shape_view#

Original source code:

fn_with_kwargs#

Original source code:

list_contains#

Original source code:

list_unpack#

Original source code:

nested_function#

Original source code:

null_context_manager#

Original source code:

pytree_flatten#

Original source code:

scalar_output#

Original source code:

specialized_attribute#

Original source code:

static_for_loop#

Original source code:

static_if#

Original source code:

tensor_setattr#

Original source code:

type_reflection_method#

Original source code:

user_input_mutation#

Original source code:

dynamic_shape_round#

Original source code:

model_attr_mutation#

Original source code:

optional_input#

Original source code:

unsupported_operator#

Original source code:
